

STEP-BY-STEP GUIDE

Publishing Your Portfolio Dashboard with GitHub Pages

A friendly, no-jargon walkthrough for putting the web app you built with Claude Code onto the internet — for free — so you (or anyone you share the link with) can open it from any browser.

Time needed: about 15–20 minutes, most of it waiting

Cost: free (GitHub Pages has no charge for a project like this)

Experience needed: none — every command is written out exactly, ready to copy and paste

Before you start, make sure you have:

- The Portfolio Dashboard project folder that Claude Code built for you, on your computer
- An internet connection
- A free GitHub account — if you don't have one yet, Step 1 covers creating it

What You're About to Do

Two minutes of plain-English background before we start clicking.

GitHub is a free website where code projects like yours are stored. **GitHub Pages** is a free feature of GitHub that takes a project and turns it into a real, working website anyone can visit — no separate hosting company, no credit card, no monthly bill.

Here's the shape of what you'll do: you'll put your project's code on GitHub (this is called "pushing"), flip one setting to turn Pages on, and GitHub will automatically build your app and publish it at an address that looks like:

```
https://your-username.github.io/portfolio-dashboard/
```

From then on, any time you make a change with Claude Code and push it again, GitHub quietly rebuilds and republishes the site on its own — usually within about a minute. You never have to repeat the setup.

A NOTE ON YOUR FINANCIAL PRIVACY

Publishing this project's *code* does **not** publish any of your money data. This app was specifically built so that your CAS statement is read and calculated entirely inside your own browser — it's never uploaded anywhere, whether the app is running on your computer or published on GitHub Pages. What you're putting on GitHub is the empty calculator, not anything you've calculated with it.

1 Create a free GitHub account

Skip to Step 2 if you already have a GitHub account and know your username and password.

1. Open **github.com/signup** in your browser.
2. Enter an email address, create a password, and choose a username. Your username becomes part of your website's address later, so pick one you're comfortable with (it doesn't have to relate to this project).
3. Follow GitHub's on-screen verification steps (usually a quick puzzle and a confirmation email).
4. Once you're signed in, you're done with this step — there's nothing to install.

TIP

Write your username and password down somewhere safe for a moment — you'll use the username again in Step 6.

2

Create a new, empty repository

A "repository" (or "repo" for short) is just GitHub's word for a project folder. You'll create an empty one now, then fill it with your project in Step 3.

1. Go to **github.com/new** (while signed in).
2. Under **Repository name**, type `portfolio-dashboard` — or any name you like; just remember it, since it becomes part of your website's address.
3. Leave it set to **Public**. (Public just means the code is visible to anyone who looks for it — remember, that's only the app itself, never your financial data. Public is also the simplest option and what this guide assumes.)
4. **Do not** check any of the boxes offering to add a README, .gitignore, or license — your project already has these. Adding them here would create conflicts in Step 3.
5. Click the green **Create repository** button.

WHAT YOU'LL SEE

Repository name

☒ Public ☐ Private

Initialize with

(leave everything here unchecked)

Create repository

CHECKPOINT

After clicking Create, GitHub shows a page titled "Quick setup" with a web address ending in `.git`. Keep this page open — you'll copy that address in the next step.

3

Send your project's code to GitHub

Now you'll connect the project folder on your computer to the empty repository you just made, and upload ("push") your code to it. Open the same terminal window you've been using with Claude Code, and make sure you're in your project's folder.

Copy the web address from the "Quick setup" page (it looks like `https://github.com/your-username/portfolio-dashboard.git`), then run these two commands, replacing the address in the first one with your own:

```
git remote add origin https://github.com/your-username/portfolio-dashboard.git
git push -u origin master
```

IF A BROWSER WINDOW POPS UP

That's expected and normal — it's GitHub asking you to confirm it's really you. Log in (or click "Authorize") in that window, then return to your terminal; the upload will continue automatically.

IF IT SAYS "MASTER" DOESN'T MATCH

Some newer setups use the name `main` instead of `master` for the same thing. If the first command fails or GitHub's own instructions on the Quick Setup page show `main`, just use that word in place of `master` above — either name works with the rest of this guide.

CHECKPOINT

Refresh the GitHub page from Step 2 — you should now see all of your project's files and folders listed there instead of the empty "Quick setup" screen.

4

Turn on GitHub Pages

This is the one-time setting that tells GitHub "yes, publish this as a website."

1. On your repository's page, click the **Settings** tab (top of the page, gear icon).
2. In the left-hand menu, click **Pages**.
3. Under **Build and deployment**, find the **Source** dropdown. It probably says "Deploy from a branch" — change it to **GitHub Actions**.
4. That's it — this saves itself, there's no separate Save button.

WHAT YOU'LL SEE — SETTINGS > PAGES

General

Build and deployment

Collaborators

Source

GitHub Actions ▼

Pages

Actions

Webhooks

You won't see a live link here yet — that appears once the first deployment finishes in the next step.

5

Watch GitHub build and publish your site

Pushing your code in Step 3 already told GitHub to start building your website automatically — you can watch it happen.

1. Click the **Actions** tab on your repository's page.
2. You'll see a run named "**Deploy to GitHub Pages**" with a yellow dot, meaning it's in progress.
3. Wait about a minute or two. When it finishes successfully, the yellow dot turns into a green checkmark.

WHAT YOU'LL SEE — ACTIONS TAB

✓ Deploy to GitHub Pages — build

✓ Deploy to GitHub Pages — deploy

IF YOU SEE A RED X INSTEAD

Click into that run to read what went wrong, then see the **Troubleshooting** section near the end of this guide.

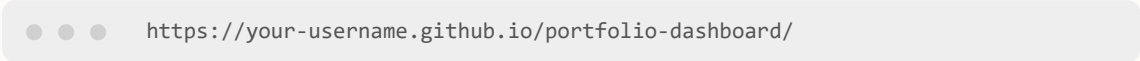
6

Visit your live website

Once both steps in Step 5 show green checkmarks, your site is live at:

```
https://your-username.github.io/portfolio-dashboard/
```

Using your own GitHub username and the exact repository name you chose in Step 2. You can also find this same address by going back to **Settings > Pages** — it now shows a "Visit site" link at the top.



https://your-username.github.io/portfolio-dashboard/

Your Portfolio Dashboard loads here, exactly as it looked when you tested it locally.

CHECKPOINT

The page should load with the sample portfolio already showing. Try dropping a real CAS statement in the upload box to confirm everything works end to end.

What's Slightly Different on GitHub Pages

Two small, honest caveats — neither affects the core dashboard.

GitHub Pages only hosts the website itself; it can't run the two small helper services this app can optionally use when hosted elsewhere. The app was built to handle this gracefully, so nothing breaks — you just get slightly fewer bells and whistles:

Feature	What happens on GitHub Pages
Today's fund prices	The app normally tries a few different public sources for up-to-date prices. One of those (routed through a small helper) isn't available on GitHub Pages, so it automatically uses its other, equally reliable sources instead. You're very unlikely to notice any difference.
The Feedback form	Sending feedback from inside the app needs a small helper to relay the message, which GitHub Pages doesn't provide. If you use the form, it will show a polite error instead of sending. Everything else in the app is unaffected.

If either of these ever matters to you, the project's `docs/DEPLOY.md` file covers an alternative host (Cloudflare Pages) that supports both — but it requires a Cloudflare account and a couple of extra setup steps, so it's optional and not needed to enjoy the dashboard.

Publishing Future Changes

You only do Steps 1–4 once. From now on, this is all it takes.

Any time you ask Claude Code to change something and it commits the result, just run one command from your project folder:

```
git push
```

GitHub automatically notices the update, rebuilds your site, and republishes it — usually within a minute or two. You can watch it happen the same way as Step 5, on the **Actions** tab, but you don't have to; it just happens in the background.

If Something Doesn't Look Right

The site shows a blank white page

Almost always means the page loaded before GitHub finished publishing, or the web address doesn't exactly match your username and repository name (this address is case-sensitive). Double-check the address against Step 6, wait a minute, and refresh.

GitHub itself shows "404 – File not found"

This is GitHub's own error page, not the app's — it means Pages isn't turned on yet (revisit Step 4) or the address is missing the repository name at the end (it must end in a `/` after your repository name).

The Actions tab shows a red X

Click into the failed run to see the error message in red text. The most common cause for a project like this is a small typo introduced by hand-editing a file outside of Claude Code. Copy the error message, paste it back to Claude Code, and ask it to fix the problem — then run `git push` again once it's fixed.

Pushing asks for a password and then fails

GitHub no longer accepts your regular account password from the command line. A browser window should pop up automatically asking you to log in instead (see the note under Step 3) — if it doesn't appear, search "GitHub personal access token" in GitHub's own Help docs for the alternative sign-in method, or ask Claude Code to walk you through it.

Quick Reference

Everything above, condensed — for once you've done this the first time.

Task	Where / What
------	--------------

Create the empty project on GitHub	<code>github.com/new</code>
Connect and upload your code (once)	<code>git remote add origin <your repo URL></code> <code>git push -u origin master</code>
Turn on Pages (once)	Settings › Pages › Source › GitHub Actions
Publish future changes	<code>git push</code>
Watch a deploy in progress	Actions tab
Your live website	<code>https://your-username.github.io/portfolio-dashboard/</code>

This guide was generated for the Portfolio Dashboard project. If the project's structure changes significantly in the future, ask Claude Code to check whether these steps still apply.